03

SKT K-뉴딜 아카데미 · AI· 자동화 기초

예외처리 로깅

깨진 줄이 섞여 있어도 멈추지 않고,
무슨 일이 있었는지 파일에 남깁니다.

1교시 — 시작 전에 한 가지만 답해 주세요

익명입니다. 답에 따라 이번 주 진도와 아침 30분의 속도를 조절합니다.

여기로 들어옵니다



https://qr-chat.physical-spark.com/r/6677192

무엇을 묻나

- 깃허브 · 리눅스 명령어를 어디까지 해 봤나
- 지난주 복습을 했나 · 문제를 어디까지 풀었나
- 지금 난이도는 어떤가 · 어떤 방식이 도움이 되나

1교시 — 스스로 찾아서 공부합니다

오늘은 깃배시를 깔고 명령 세 개를 쳐 봅니다. 설명은 없습니다 — 찾아서 하는 것까지가 과제입니다.

1 · 과제 문서를 엽니다

단톡방에 올린 파일을 엽니다 — `0928_morning30.md`

2 · 오늘 할 일 셋

왜 리눅스와 깃허브를 배우는지 찾아 적습니다

깃배시를 설치합니다 — 고를 것이 정해져 있습니다

`pwd` · `ls` · `ls -a` 를 쳐 봅니다

다 못 해도 됩니다.
오후 5시 이후에 마무리하면 되고, 어디서 멈췄는지만 적어 둡니다.

깃배시 설치 — 이 두 개만 바꿉니다

선택 화면이 열 번쯤 나옵니다. 둘만 바꾸고 나머지는 「Next」입니다.

설치 화면 제목	고를 것	왜
Choosing the default editor	Notepad	기본값이 Vim 이라 처음 보면 닫는 법을 몰라 막힙니다
Adjusting the name of the initial branch	main	10/7 에 폴더를 저장소와 이을 때 이름이 같아야 합니다

설치가 끝나면 시작 버튼에 `git bash` 를 입력합니다.
창이 열리고 `$` 가 보이면 정상입니다.

오늘 오전에 log_parser.py 를 만듭니다

깨진 줄이 섞인 파일을 넣어도 화면 결과가 똑같이 나오고, 기록이 남습니다.

> 기록은 발췌입니다 — 깨진 줄이 5건이라 경고도 5줄입니다.

```
# (화면)
----------------------------------------------------------------------
확인 필요: admin — 실패 3회

# (agent.log)
----------------------------------------------------------------------
2026-09-28 12:40:01 INFO 파서 시작: sample_logs_broken.csv
2026-09-28 12:40:01 WARNING 깨진 줄 건너뜀: 03:22,hacker
2026-09-28 12:40:01 INFO 정상 로그 17건 처리 완료
```

이 시간에 나오는 낱말 세 개

용어 (한글발음)	한 줄 뜻	오늘 어디서 만나나
예외 exception, 익셉션	실행 도중 생긴 문제로 프로그램이 멈추는 것	칸이 모자란 로그 줄을 읽을 때
try / except 트라이 엑셉트	문제가 날 만한 줄을 감싸 두고 대비하는 문장	`log_parser.py` 의 읽는 반복문
트레이스백 traceback, 트레이스백	멈춘 자리까지 어떤 줄을 거쳤는지 적어 주는 화면	파일 이름을 틀렸을 때 뜨는 화면

> **문법 오류는 실행 전에, 예외는 실행 중에 납니다.**

상황: 새벽 여섯 시에 저절로 도는 프로그램

어느 날 로그 파일 이름이 바뀌어, 파일을 못 찾고 멈췄습니다.

사람이 옆에 있다면 대응할 수 있다.

- 화면에 트레이스백이 뜬다
- 보고 바로 고친다
- 그날 점검은 그대로 끝난다

하지만 사람이 아무도 없을 때

- 트레이스백이 떠도 고칠 사람이 없다
- 그날 점검이 통째로 비어 있다
- 비었다는 사실조차 모른다

자동화에서 제일 위험한 것은
조용한 실패입니다

틀린 답보다 무서운 것은, 아무 답도 없는데 아무도 모르는 상태입니다.

① **멈추지 않게 만든다**
— `try · except`

② **무슨 일이 있었는지 남긴다**
— `logging`

③ **둘을 묶는다**
— 4교시에 `log_parser.py`

오늘 쓸 파일은 두 개입니다

깨진 줄이 5개 섞여 있습니다 — 칸이 모자란 줄, 빈 줄, 로그가 아닌 안내문.

sample_logs.csv — 정상 17줄

```
09:01,kim01,LOGIN_OK,10.0.3.21
09:03,lee02,LOGIN_FAIL,10.0.7.5
09:05,admin,LOGIN_FAIL,10.0.9.8
```

sample_logs_broken.csv — 21줄

```
09:01,kim01,LOGIN_OK,10.0.3.21
03:22,hacker
09:05,admin,LOGIN_FAIL,10.0.9.8
```

파일 이름이 하나 다르면 여기서 멈춥니다

아래 두 줄 중 둘째 줄은 실행되지 않습니다.

> 「읽기 시작」이 안 찍혔습니다. 멈춘 자리 아래는 아무것도 실행되지 않습니다.

```
f = open("sample_logs.csv")
print("읽기 시작")
```

```
Traceback (most recent call last):
  File "a.py", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'sample_logs.csv'
```

트레이스백은 맨 아래부터 읽습니다

위는 거쳐 온 길이고, 답은 마지막 줄에 있습니다.

줄	무엇이 적혀 있나	언제 보나
Traceback …	아래에 오류 설명이 있다는 안내	읽지 않아도 된다
File … line 1	몇 번째 줄에서 멈췄는지	코드가 길 때
f = open(…)	멈춘 그 줄을 그대로 보여 준다	어느 줄인지 눈으로 확인
FileNotFoundError: …	오류 이름과 이유 — 여기가 답이다	항상 먼저

로그를 다룰 때 자주 멈추는 세 가지

무엇을 했나	멈출 때 나오는 이름과 이유
없는 파일을 연다 `open("sample_logs.csv")`	**`FileNotFoundError`** No such file or directory: 'sample_logs.csv'
칸이 모자란 줄에서 네 번째 칸을 꺼낸다 `parts[3]`	**`IndexError`** list index out of range
숫자로 바꿀 수 없는 글자를 숫자로 바꾼다 `int("hacker")`	**`ValueError`** invalid literal for int() with base 10: 'hacker'

셋 다 문법은 맞습니다. 외우지 않아도 되고, 보고 알아보면 됩니다.

try · except 는 이렇게 생겼습니다

적는 자리가 셋입니다. 지난주 `if` 와 규칙이 같습니다.

> 두 줄 다 끝에 콜론이 붙고, 그 아래는 네 칸 들여씁니다.

```
try:
    # 해 볼 일 — 문제가 날 만한 줄
    f = open("sample_logs.csv")
except FileNotFoundError:
    # 그 예외가 났을 때 대신 할 일
    print("파일을 찾을 수 없습니다")
```

적는 자리가 셋입니다

방금 본 여섯 줄에서 어디에 무엇을 적는지 나눠 봅니다.

적는 자리	무엇을 적나
`try:` 아래	해 볼 일 — 문제가 날 만한 줄만 넣습니다
`except` 뒤	잡을 예외 이름 — 파일이 없을 때는 `FileNotFoundError`
`except:` 아래	그 예외가 났을 때 대신 할 일

`try` 로 감싸면 프로그램이 멈추지 않고 넘어갑니다

> 문제를 없앤 게 아니라 멈추지 않게 대비한 것입니다.

감싸지 않으면

```
f = open("sample_logs.csv")
print("점검 계속")
```

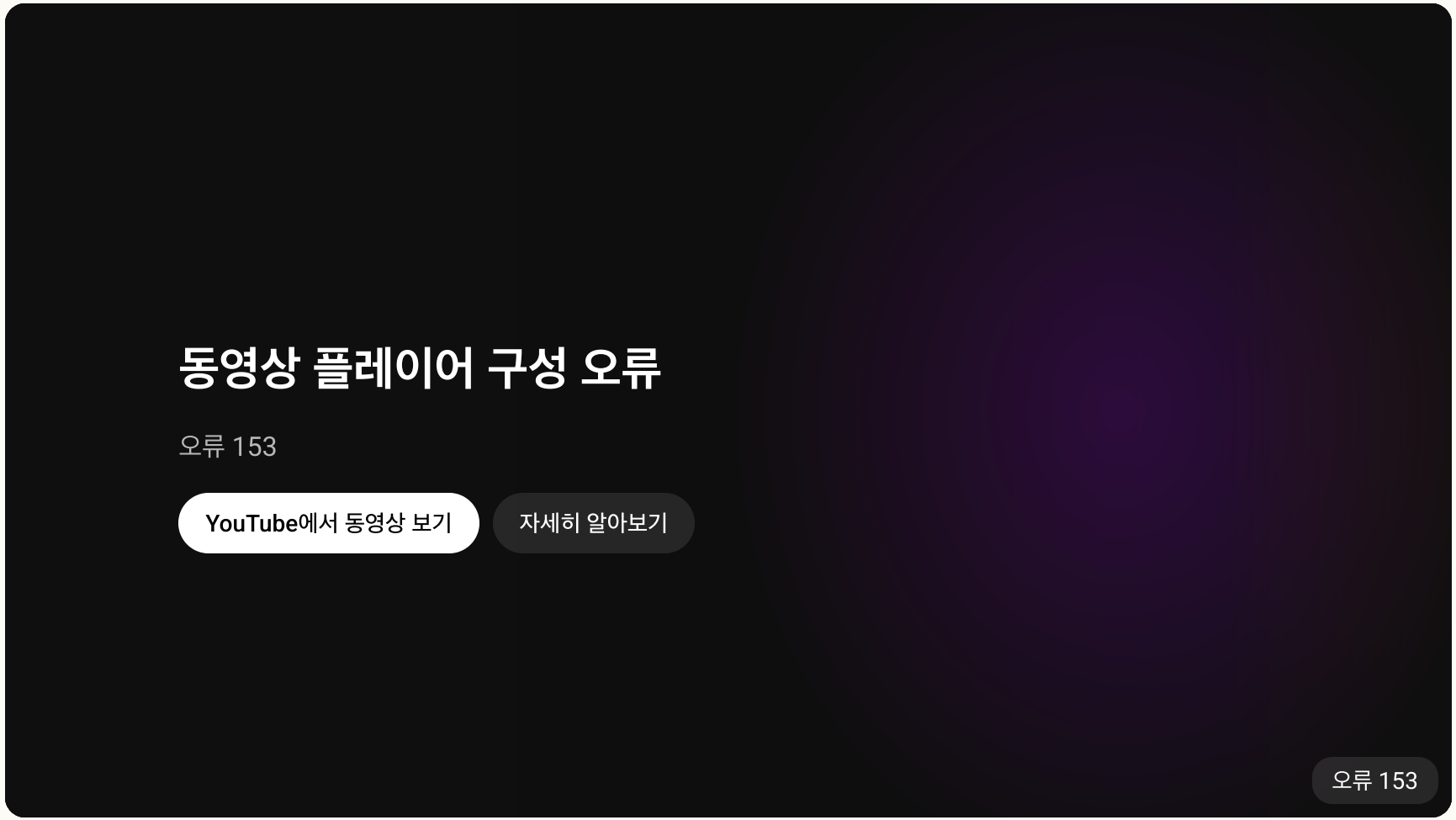
```
FileNotFoundError
(「점검 계속」은 안 찍힘)
```

try 로 감싸면

```
try:
    f = open("sample_logs.csv")
except FileNotFoundError:
    print("파일을 찾을 수 없습니다")
print("점검 계속")
```

```
파일을 찾을 수 없습니다
점검 계속
```

3분 영상 — 에러가 나도 멈추지 않게



깨진 한 줄 때문에 남은 줄을 못 읽습니다

로그 한 줄은 `시각,계정,사건,IP` 네 칸입니다. 셋째 줄만 쉼표 뒤가 잘려 두 칸입니다.

> 넷째 줄 admin LOGIN_FAIL 은 읽히지 않았습니다. 뒤에 정상인 줄이 천 개 있어도 마찬가지입니다.

```
lines = ["09:01,kim01,LOGIN_OK,10.0.3.21",
         "09:12,lee02,LOGIN_FAIL,10.0.3.77",
         "03:22,hacker",
         "03:25,admin,LOGIN_FAIL,45.9.12.3"]

for line in lines:
    parts = line.split(",")
    print(parts[1], parts[2])
```

```
kim01 LOGIN_OK
lee02 LOGIN_FAIL
IndexError: list index out of range
```

try 를 반복문 안에 넣으면 그 줄만 건너뜁니다

한 바퀴가 실패해도 그 바퀴만 except 로 가고, 다음 바퀴는 그대로 돕니다.
노트북에서는 21줄짜리 파일에 돌려 깨진 다섯 줄을 건너뛰고 정상 17건을 읽습니다.

```
lines = ["09:01,kim01,LOGIN_OK,10.0.3.21",
         "09:12,lee02,LOGIN_FAIL,10.0.3.77",
         "03:22,hacker",
         "03:25,admin,LOGIN_FAIL,45.9.12.3"]

for line in lines:
    parts = line.split(",")
    try:
        print(parts[1], parts[2])
    except IndexError:
        print("건너뜀:", line)
```

```
kim01 LOGIN_OK · lee02 LOGIN_FAIL
건너뜀: 03:22,hacker
admin LOGIN_FAIL
```

읽은 줄과 건너뛴 줄을 숫자로 세어 둡니다

> `ok` 는 출력이 성공한 다음 줄에 있어 실패한 바퀴에는 늘지 않습니다.
> 「스무 줄 중 하나를 버렸다」와 「열여덟을 버렸다」는 전혀 다른 상황이라 이 숫자가 필요합니다.

```
lines = ["09:01,kim01,LOGIN_OK,10.0.3.21",
         "09:12,lee02,LOGIN_FAIL,10.0.3.77",
         "03:22,hacker",
         "03:25,admin,LOGIN_FAIL,45.9.12.3"]

ok = 0
skipped = 0
for line in lines:
    try:
        print(line.split(",")[2])
        ok = ok + 1
    except IndexError:
        skipped = skipped + 1
print("읽은 줄:", ok, "· 건너뛴 줄:", skipped)
```

```
LOGIN_OK
LOGIN_FAIL
LOGIN_FAIL
읽은 줄: 3 · 건너뛴 줄: 1
```

try 안에 여러 줄이면 어디서 멈출까요

칸이 두 개뿐인 줄입니다. 화면에 몇 줄이 찍힐지 먼저 세어 보세요.

```
parts = "03:22,hacker".split(",")
try:
    print("계정:", parts[1])
    print("사건:", parts[2])
    print("여기까지 올까")
except IndexError:
    print("건너뜀")
print("다음 단계로")
```

예외가 난 아래 줄은 사라집니다

앞 장과 같은 코드입니다. 화면에 세 줄이 찍힙니다.

> `parts[2]` 에서 멈췄으니 그 아래 「여기까지 올까」는 영영 실행되지 않습니다.
> 그래서 `try` 안에는 문제가 날 만한 줄만 넣습니다.

```
parts = "03:22,hacker".split(",")
try:
    print("계정:", parts[1])
    print("사건:", parts[2])
    print("여기까지 올까")
except IndexError:
    print("건너뜀")
print("다음 단계로")
```

```
계정: hacker
건너뜀
다음 단계로
```

정리 — `try` 로 감싼다는 것은 멈추지 않게 만드는 것입니다

① **`try:` 아래 — 해 볼 일**

문제가 날 만한 줄만 넣습니다. 멀쩡한 줄까지 다 넣으면 어디가 문제인지 흐려집니다

② **`except:` 아래 — 대신 할 일**

건너뛰거나, 기본값을 쓰거나, 남깁니다. 비워 두면 조용한 실패가 됩니다

③ **그 뒤 — 프로그램은 계속됩니다**

오류가 사라진 것이 아니라, 멈추지 않게 대비한 것입니다

실습 ① · 노트북 1장 · 01_0928_am_exceptions_logging.ipynb

문제 2-1 ~ 2-10 · 도전 ⭐2-1 ~ ⭐2-5

① **사본을 저장하고 맨 위 셀을 실행합니다**

파일 › 드라이브에 사본 저장 → 오늘 쓸 로그 파일 두 개가 만들어집니다

② **문제 2-1 · 1-2 — 무엇이 보일까요**

실행하기 전에 화면에 무엇이 나올지 먼저 적어 봅니다

③ **문제 2-3 ~ 2-10 을 풉니다**

깨진 파일을 끝까지 읽고 「건너뜀」이 찍히면 성공

이 시간에 나오는 낱말 세 개

용어 (한글발음)	한 줄 뜻	오늘 어디서 만나나
ValueError 밸류 에러	숫자로 바꿀 수 없는 글자를 넘겼을 때 나는 예외	`int("unknown")` 을 실행할 때
FileNotFoundError 파일 낫 파운드 에러	그 경로에 파일이 없을 때 나는 예외	파일 이름을 한 글자 틀렸을 때
finally 파이널리	성공하든 실패하든 마지막에 꼭 하는 묶음	열어 둔 파일을 닫는 자리

「건너뜀 5건」만으로는 왜 깨졌는지 모릅니다

깨진 줄을 모두 `IndexError` 로 묶어 버렸습니다.

한 가지로 묶으면

- 「건너뜀 5건」만 남는다
- 무엇이 왜 깨졌는지 모른다
- 고칠 곳을 찾을 수 없다

이름으로 나눠 잡으면

- 「칸 모자람 3건 · 시각 깨짐 2건」
- 깨진 이유마다 다르게 대응한다
- 어느 시스템이 잘못 쓰는지 보인다

로그가 깨지는 방식은 여럿입니다

깨진 줄	무엇이 문제인가	멈출 때 나오는 이름
`02:40,hacker`	칸이 두 개뿐이다	`IndexError`
`1a:07,gate01,…`	시각에 글자가 섞였다	`ValueError`
`서버 점검 안내: …`	로그가 아닌 줄이 섞였다	`IndexError`

세 번째 줄은 쉼표가 없어 칸이 하나뿐이고, 그래서 칸 모자람으로 잡힙니다.

오류 이름은 그대로 검색어입니다

모르는 이름이 떠도 막히지 않습니다. 이름을 그대로 붙여 넣으면 됩니다.

화면에 뜬 것	검색창에 넣을 것
`FileNotFoundError: [Errno 2] No such file…`	`python FileNotFoundError`
`ValueError: invalid literal for int()…`	`python ValueError invalid literal for int`

> 계정 이름·파일 경로 같은 내 값은 빼고 넣습니다.
> 그 부분은 사람마다 달라서 검색에 방해가 됩니다.

판단하려면 시각을 숫자로 바꿔야 합니다

"23" 은 글자입니다. 크기를 비교하려면 숫자여야 합니다.

쉼표로 나눈 뒤 콜론으로 한 번 더 나눕니다.
그 첫 칸이 시(時)입니다.
왜 굳이 숫자로 바꾸나 — 글자끼리 비교하면 "9" 가 "23" 보다 크다고 나옵니다.
크기를 따지려면 숫자여야 합니다.

```
line = "23:50,gate01,LOGIN_FAIL,10.0.5.9"
parts = line.split(",")
hour = int(parts[0].split(":")[0])
print(hour)
```

```
23
```

시각 칸에 글자가 섞이면 멈춥니다

메시지 끝의 `'1a'` 가 문제가 된 값입니다. 오류 메시지는 어떤 값 때문인지까지 알려 줍니다.

```
line = "1a:07,gate01,LOGIN_FAIL,10.0.5.9"
hour = int(line.split(",")[0].split(":")[0])
```

```
ValueError: invalid literal for int() with base 10: '1a'
```

시각이 숫자가 되면 야간인지 판단할 수 있습니다

0시부터 6시 사이면 야간 접속으로 봅니다. 기준은 사람이 정합니다.

> 지난주 `if` 와 `and` 그대로입니다. 새로 붙은 것은 시각을 숫자로 만든 앞 줄뿐입니다.

```
hour = 3
if hour >= 0 and hour <= 6:
    print("야간 접속")
else:
    print("주간")
```

```
야간 접속
```

ValueError 는 로그의 어디에서 나는지 확인합니다

로그의 칸	무엇을 하려다	깨진 값의 예
시각	시를 숫자로 바꾸려다	`1a:07 · --:--`
실패 횟수	숫자로 세려다	`N/A · 빈칸`
포트 번호	숫자로 비교하려다	`unknown`

> 공통점이 있습니다 — 글자를 숫자로 바꾸는 자리입니다. 그 줄을 `try` 로 감쌉니다.

except 를 여러 개 이어 쓰면 맞는 것 하나만 돕니다

```
lines = ["23:14,kim01,LOGIN_OK,10.0.5.9",
         "1a:07,gate01,LOGIN_FAIL,10.0.5.9", "02:40,hacker"]

for line in lines:
    parts = line.split(",")
    try:
        print(int(parts[0].split(":")[0]), parts[3])
except ValueError:
    print("시각 깨짐:", line)
except IndexError:
    print("칸 모자람:", line)
```

세 줄이 각각 다른 곳으로 갔습니다

```
23 10.0.5.9
시각 깨짐: 1a:07,gate01,LOGIN_FAIL,10.0.5.9
칸 모자람: 02:40,hacker
```

줄	어디로 갔나
첫째	정상 — 끝까지 돌았습니다
둘째	시각이 깨져 ValueError 쪽으로
셋째	칸이 모자라 IndexError 쪽으로

Exception 을 위에 두면 아래 except 는 안 돕니다

Exception 은 모든 예외를 잡는 이름입니다.
위에 두면 아래에 적은 ValueError 까지 전부 가로챕니다.
지난주 elif 와 같습니다 — 위에서부터 보다가 처음 맞는 하나만 실행됩니다.

Exception 을 위에 두면

```
try:
    hour = int("1a:07".split(":")[0])
except Exception:
    print("뭔가 잘못됐다")
except ValueError:
    print("시각이 깨졌다")
```

```
뭔가 잘못됐다
(ValueError 쪽은 영영 안 돈다)
```

정해진 이름을 위에 두면

```
try:
    hour = int("1a:07".split(":")[0])
except ValueError:
    print("시각이 깨졌다")
except Exception:
    print("뭔가 잘못됐다")
```

```
시각이 깨졌다
```

finally 에는 무엇을 넣나

`finally` 아래 줄은 성공하든 예외가 나든 마지막에 반드시 실행됩니다. `try` · `except` 뒤에 한 번만 씁니다.

> 성공한 경우에만 닫으면, 실패했을 때 파일이 열린 채 남는데,
> 파일이 쌓이면 프로그램이 파일을 더 못 엽니다.
> 이 일을 자동으로 해 주는 `with` 라는 문법이 따로 있습니다 (오늘 오후에 파일을 저장할 때 봅니다.)

```
f = open("sample_logs.csv")
try:
    rows = f.read().splitlines()
except Exception:
    rows = []
finally:
    f.close()          # 성공하든 실패하든 닫는다
```

성공한 줄에서도 finally 가 돕니다

23 을 찍고 나서 「한 줄 끝」이 또 나왔습니다.

```
t = "23:50"
try:
    print(int(t.split(":")[0]))
except ValueError:
    print("읽을 수 없음")
finally:

    print("--- 한 줄 끝")
```

```
23
--- 한 줄 끝
```

실패한 줄에서도 finally 가 돕니다

값만 바꿨는데 「한 줄 끝」은 그대로 나왔습니다 — 열어 둔 것을 닫거나 임시 파일을 지우는 자리입니다.

```
t = "1a:07"
try:
    print(int(t.split(":")[0]))
except ValueError:
    print("읽을 수 없음")
finally:

    print("--- 한 줄 끝")
```

```
읽을 수 없음
--- 한 줄 끝
```

이름 없는 except: 는 쓰지 않습니다

> 왼쪽은 parts 를 part 로 잘못 적은 진짜 버그까지 숨깁니다.
> 2교시의 「조용한 실패」를 우리가 만드는 셈입니다.

이름을 안 적으면

```
try:
    hour = int(part[0])
except:
    pass
```

변수 이름 오타까지 조용히 삼킨다

이름을 적으면

```
try:
    hour = int(part[0])
except ValueError:
    print("시각이 깨졌다")
```

```
NameError: name 'part' is not defined
```

파일이 없으면 빈 목록을 돌려줍니다

try 로 파일을 엽니다. 열렸느냐에 따라 갈라집니다.

열렸다

읽은 줄을 목록에 담는다

↓

목록을 돌려준다

파일이 없다

except FileNotFoundError
경로 문제라고 남긴다

↓

빈 목록을 돌려준다

멈추지 않고 빈 목록을 넘깁니다. 뒤에 붙은 알림과 보고서가 살아 있어야 하기 때문입니다.

실습 ② · 노트북 3교시 절 · 01_0928_am_exceptions_logging.ipynb

문제 3-1 ~ 3-10 · 도전 ⭐3-1 ~ ⭐3-5

① **문제 3-1 · 2-2 — 무엇이 보일까요**

실행하기 전에 화면에 무엇이 나올지 먼저 적어 봅니다

② **문제 3-3 ~ 3-7 — 시각을 숫자로, 야간을 가려내기**

`21:30` 에서 `21` 이 나오고, `03:22` 가 야간으로 잡히면 성공

③ **문제 3-8 ~ 3-10 — 이유를 나눠 세기**

깨진 줄을 시각 깨짐·칸 모자람으로 나눠 세면 성공

이 시간에 나오는 낱말 두 개

용어 (한글발음)	한 줄 뜻	오늘 어디서 만나나
로깅 logging, 로깅	프로그램이 한 일을 시각과 함께 남기는 것	`agent.log` 에 쌓이는 줄
로그 레벨 level, 레벨	로그의 심각도 등급	`INFO` · `WARNING` · `ERROR`

> 이 교시가 끝나면 agent_core/log_parser.py 와 로그 파일 agent.log 가 남습니다.

print 로 찍은 글자는 창을 닫으면 사라집니다

print 로 찍으면

- 화면에만 나온다
- 창을 닫으면 없어진다
- 새벽에 돈 결과는 볼 수 없다

로깅으로 남기면

- 파일에 줄로 쌓인다
- 시각과 등급이 함께 남는다
- 아침에 열어 보면 된다

%(asctime)s 자리에 시각이 채워집니다

```
format="%(asctime)s %(levelname)s %(message)s"

2026-09-28 12:40:01,082 INFO 파서 시작
```

적는 것	무엇이 채워지나
%(asctime)s	로그를 남긴 시각
%(levelname)s	등급 — INFO · WARNING · ERROR
%(message)s	우리가 적은 말

> 괄호 안 이름은 파이썬이 정해 둔 것이라 그대로 적습니다.

`basicConfig` — 맨 위에 한 번만 적는 설정 네 칸

어디에 남길지 · 어느 등급부터 · 어떤 모양으로 · 어떤 인코딩으로. 네 칸을 한 번 정해 두면 그다음은 `logging.info(...)` 한 줄이면 됩니다.

> 실행해도 화면에는 아무것도 안 나옵니다 — 로그가 파일로 갔기 때문입니다.
> 설정은 프로그램 맨 위에 한 번만 적습니다.

```
import logging

logging.basicConfig(
    filename="agent.log",           # 어느 파일에
    level=logging.INFO,             # 어느 등급부터
    format="%(asctime)s %(levelname)s %(message)s",   # 한 줄을 어떤 모양으로
    encoding="utf-8",               # 한글이 깨지지 않게
)
logging.info("파서 시작")
```

```
(화면에는 아무것도 안 나옵니다)
```

적어 둔 등급부터 오른쪽 전부가 남습니다

DEBUG
가장 가볍다

→ **INFO** → **WARNING** →

ERROR
가장 심각하다

`level` 에 적은 등급	파일에 남는 것	버리는 것
안 적으면 — 기본 WARNING	WARNING · ERROR	DEBUG · INFO
`level=logging.INFO`	INFO · WARNING · ERROR	DEBUG
`level=logging.ERROR`	ERROR	나머지 셋

기본값이 WARNING 이라 아무것도 안 적으면 `logging.info` 는 남지 않습니다.

파이썬이 정해 둔 등급은 다섯입니다

등급	숫자	오늘 파서에서
DEBUG	10	안 씁니다 — 개발 중에 보는 것
INFO	20	정상으로 지나간 일
WARNING	30	깨진 줄 건너뜀 — 기본값
ERROR	40	파일을 못 찾았다
CRITICAL	50	안 씁니다 — 프로그램이 더 못 돌 때

우리가 정한 이름이 아닙니다. 보안 장비 표준(syslog)은 여덟 단계로, 2·3과목에서 봅니다.

level 을 안 적으면 info 가 안 남습니다

> 앞 장의 규칙을 그대로 돌려 본 것입니다 — 「info 가 안 찍혀요」의 가장 흔한 원인이 이 한 줄입니다.

level 을 안 적으면

```
logging.basicConfig(filename="a.log")
logging.info("정상으로 지나간 일")
logging.warning("봐 둘 일")
```

```
WARNING 봐 둘 일
(info 한 줄은 사라졌다)
```

level=INFO 를 적으면

```
logging.basicConfig(filename="a.log",
                    level=logging.INF
)
logging.info("정상으로 지나간 일")
```

```
INFO 정상으로 지나간 일
WARNING 봐 둘 일
```

등급은 여러 개지만 오늘은 세 개만 씁니다

등급	무엇을 남기나	오늘 파서에서 쓰는 자리
INFO	정상적으로 지나간 일	파서 시작 · 정상 17건 처리 완료
WARNING	지금은 돌지만 봐 둘 일	깨진 줄 건너뜀
ERROR	하려던 일이 실패한 것	파일을 못 찾았다

> 깨진 줄은 ERROR 가 아닙니다. 프로그램은 제 할 일을 했고, 다만 봐 둘 일이 생긴 것입니다.

로그 한 줄은 시각 · 등급 · 메시지 순서입니다

왼쪽 칸이 앞 장 `format` 에 적은 그 글자입니다.
순서를 바꾸고 싶으면 그 줄을 고치면 됩니다.
등급이 왜 따로 있는지는 다음 장에서 봅니다 — 골라 보려고 있는 것입니다.

```
# agent.log
----------------------------------------------------------------------------------------------------
2026-09-28 12:40:01,082  INFO  파서 시작: sample_logs_broken.csv
```

format 에 적은 말	이 줄에서는	무엇
`%(asctime)s`	2026-09-28 12:40:01,082	남긴 시각 — 쉼표 뒤 082 는 밀리초입니다
`%(levelname)s`	INFO	등급 — 나중에 골라 볼 때 쓴다
`%(message)s`	파서 시작: sample_logs_broken.csv	우리가 적은 말

화면과 로그을 같이 씁니다

print 를 없애는 것이 아닙니다. 둘은 읽는 사람이 다릅니다
— 화면은 지금 보는 사람에게 결과를, 로그는 나중에 보는 사람에게 과정을 줍니다.
그래서 화면에는 한 줄만 찍었습니다.
깨진 줄을 다 찍으면 정작 봐야 할 결과가 묻힙니다.

화면 — 사람이 지금 볼 것

```
확인 필요: admin — 실패 3회
```

결과 한 줄만

agent.log — 나중에 볼 것

```
INFO 파서 시작: sample_logs_broken.csv
WARNING 깨진 줄 건너뜀: 03:22,hacker
```

지나간 과정까지

등급을 나눠 두면 골라 볼 수 있습니다

로그가 만 줄이어도 봐야 할 줄만 꺼냅니다.

> `print` 로는 이렇게 못 합니다. 등급이라는 칸이 없기 때문입니다.

```
# agent.log 에서 WARNING 만
----------------------------------------------------------------------------------------------------
2026-09-28 12:40:01 WARNING 깨진 줄 건너뜀: 03:22,hacker
2026-09-28 12:40:01 WARNING 깨진 줄 건너뜀: 09:41,guest
2026-09-28 12:40:01 WARNING 깨진 줄 건너뜀: ??:??,unknown,LOGIN_FAIL
```

같은 프로그램을 두 번 돌리면 로그가 아래에 이어 붙습니다

두 줄의 시각이 다릅니다 — 첫 실행과 두 번째 실행이 모두 남아 있습니다.

로깅은 덮어쓰지 않고 덧붙입니다.

새벽에 무슨 일이 있었는지 아침에 확인할 수 있는 이유예요.
그래서 다음 장의 문제가 생깁니다 — 덧붙이기만 하면 파일이 끝없이 커집니다.

```
# log_parser.py 를 두 번 실행한 뒤 agent.log
---------------------------------------------------------------------------
2026-09-28 12:40:01 INFO 파서 시작
2026-09-28 12:41:37 INFO 파서 시작
```

로그는 지워지지 않고 계속 쌓입니다

파일 하나가 백만 바이트가 되면 새 파일로 넘기고 지난 파일을 다섯 개까지 남깁니다 — 여섯 개째가 되면 가장 오래된 것이 지워집니다.

`maxBytes` 파일 하나의 최대 크기 · `backupCount` 지난 파일을 몇 개까지 남길지

```
# 정해진 크기가 되면 새 파일로 넘기는 핸들러 — 오늘은 눈으로만 봅니다
from logging.handlers import RotatingFileHandler

RotatingFileHandler("agent.log",
maxBytes=1_000_000
,
backupCount=5
)
```

로그에 적지 말아야 하는 것

적는다	적지 않는다
계정 이름 · 시각 · IP	비밀번호 — 틀린 것도 안 된다
깨진 줄 원문	API 키 · 토큰
처리한 건수	주민번호 · 카드번호

> **로그는 지워지지 않고 여러 사람이 봅니다.**
> **한 번 적히면 거둬들일 수 없습니다.**

log_parser.py — 완성된 모양

빈 목록을 만들고 한 줄씩 읽는 여기까지는 지난주에 한 것입니다.
2교시 코드에서 `print` 가 `logging.warning` 으로 바뀐 것 하나뿐이에요.
마지막 줄의 `f` 는 지난주에 배운 f-string — `len` 으로 센 개수를 문장에 끼워 넣습니다.

```
import logging
logging.basicConfig(filename="log_parser.log", level=logging.INFO)
logging.info("파서 시작")
logs = []
for line in open("sample_logs_broken.csv"):
    parts = line.strip().split(",")
    try:
        logs.append(parts[3])
    except IndexError:
        logging.warning(f"깨진 줄 건너뜀: {line.strip()}")
logging.info(f"정상 로그 {len(logs)}건 처리 완료")
```

결과물은 드라이브 agent_core 에 남깁니다

코랩은 연습장이고, 드라이브 `agent_core` 가 코드 보관함입니다.

> `%cd` 를 빼먹으면 파일이 코랩 안에만 남고 드라이브에는 아무것도 안 생깁니다.

```
from google.colab import drive
drive.mount("/content/drive")
%cd /content/drive/MyDrive/agent_core
# 그다음 데이터 셀과 %%writefile 셀을 다시 실행합니다
!python log_parser.py
```

```
정상 로그 17건
```

실습 ③ · 노트북 4교시 절 · 01_0928_am_exceptions_logging.ipynb

문제 4-1 ~ 4-10 · 오전 결과물 두 개

① **문제 4-1 ~ 4-5 — 로그를 남겨 본다**

`agent.log` 에 세 등급이 남고, 두 번 실행하면 아래에 이어 붙으면 성공

② **문제 4-9 — `log_parser.py` 조립**

화면에 확인 필요: admin — 실패 3회 가 나오면 성공

③ **문제 4-10 — 드라이브에 남기기**

`agent_core` 에 `log_parser.py` 와 `agent.log` 가 보이면 성공